

231275036 朱晗 作业6

23# 习题目录

- 13.1, 13.2, 13.4, 13.5, 13.6, 13.8, 13.9, 13.10, 13.11, 13.12, 13.13, 13.15, 13.16, 13.18, 13.20, 13.23, 13.24

习题

13.1

请给出最大相容任务集合问题的动态规划算法的设计与分析

定义最大相容子问题的 S_{ij} 表示在 a_i 结束之后, 在 a_j 开始之前的所有的任务。(按结束时间排序过的)

所得到的递归表达式为

$$c[i, j] = \begin{cases} 0, & \text{if } S_{ij} = \emptyset \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} \end{cases}$$

即在 S_{ij} 中选择一个任务后, 再递归的做。子问题依赖关系是依赖行更靠下, 列更靠后的。所以从下往上, 从左往右算

```
def maxTask(S):
    sort(S) by the finish time of a_i
    def isSiEmpty(i,j):
        return i>j;
    init c[][], that all c[i][j] fit isSiEmpty(i,j) to 0

    for(int i=n;i>=1;i--){
        for(int j=i;j<=n;j++){
            # 从多个k中选一个
            for(int k=i+1;k<=j-1;k++){
                c[i][j]=max{c[i][k]+c[k][j]+1}
            }
        }
    }
```

此算法需要三层循环来找到最优解, 最后直接返回 $c[0][n]$, 所以是 $O(n^3)$

13.2

整数子集问题：给定一个自然数集合 $A = \{s_1, s_2, \dots, s_n\}$ 和自然数 S ，要判断是否存在 A 的子集，其中元素的和恰好是 S 。请设计一个动态规划算法来解决。

按照每个元素选或不选作为标签，子问题设计为某序列从 1 到 i 和这个子集要达到的和 j ，可以构建为一个 $dp[i][j]$ 的数组， $dp[i][j]$ 表示这个子问题是否有解 (1/0)

转移关系为

$$dp[i][j] = \begin{cases} \exists k \leq j, dp[i-1][k] \wedge s_i = j - k \\ dp[i-1][j] \end{cases}$$

即如果前 $i-1$ 位已经能达到 j ，可以选择不选 i 元素来维持 j 。否则，则遍历 k 查看是否存在加上 i 元素能凑 j 的情况。

初始化：

$dp[i][0]=True$ $dp[i][s_i]=True$ $dp[0][j]=False$

子问题依赖：依赖 i 更小， j 更小的问题。所以从上往下从左往右算

```
initialize dp[i][j], dp[i][0]=1, dp[i][s_i]=1, dp[0][j]=False, dp[0][j>S]=False
initialize all other dp[i][j]=0
def subSetSum(A,S):
    for(int i=1;i<=n;i++){
        for(int j=0;j<=S;j++){
            if(dp[i-1][j]==True):
                dp[i][j]=True

            else:
                for k in range(1,j):
                    if dp[i-1][k] and s_i=j-k
                        dp[i][j]=1
        }
    }
    return dp[n][S] #即为答案
```

13.4

给定一个自然数数组 $A[1 \dots n]$ ，请设计一个动态规划算法计算 A 中最长非递减子序列（子序列元素在数组 A 中不一定是连续出现的）

解法：由于非递减子序列的更新，只取决于一个序列的最后一个元素的大小。所以这样划分子问题

$dp[i][j]$

- i 表示从序列 $[1, \dots, i]$ 的子序列
- j 表示以第 j 个元素结尾。
- $dp[i][j]$ 表示在以上条件下的最长非递减子序列的长度
首先初始化 $dp[1][1] = 1$
不难列出 dp 方程

$$dp[i][j] = \{\max_j\{dp[i-1][j] + I_{A[i] \geq A[j]} \text{ (指示变量)}\}\}$$

如果不是新的 i, j , 则记录 $dp[i][j] = dp[i-1][j]$

最后遍历 $dp[n][j]$ 找最大值即可。

13.5

$X = x_1, x_2, \dots, x_n$ 为正整数序列。序列 X 的一个 k -划分指的是将 X 分成 k 个不重叠的连续子序列。代价 $C(P)$ 定义为划分中连续子序列和的最大值。给定参数 k , 设计一个算法计算所有可能的划分中的最低代价。

- 首先为每一位元素计算前缀和数组 $A[i] = \sum_{i=1}^n x_i$ (方便计算子序列和)
- 子问题可以划分为 $dp[i][j][k]$, 即对 $i \dots j$ 元素进行 k 划分的代价
- 递归关系为

$dp[i][j][k] = \min(\max(dp[i][j-m][k-1], \text{sum}(j-m+1, j)))$ 即递归递归计算在之前 $j-m$ 做 $k-1$ 划分和本划分 $(j-m+1, j)$ 的代价谁更大, 谁更大就是总体的代价。然后在最大的代价中取最小

依赖关系: 依赖更小的 j , 更小的 k 。所以 $[j][k]$ 从小到大计算。(以上的 i 实际上没有用到)

$\text{sum}[j-m+1, j]$ 可以通过 $A[j] - A[j-m]$ 在 $O(1)$ 内得到

```
def minPartialCost(X,A,max_k):
    init dp[j][k]
    dp[j][1]=A[j]
    for(int j=1;j<=n;j++){
        for(int k=1;k<=max_k;k++){
            min_cost=inf
            for(int m=1;m<=j-1;m++){
                tmp=max(dp[j-m][k-1],sum(j-m+1,j))
                if tmp<min_cost:
                    min_cost=tmp
            }
            dp[j][k]=min_cost
        }
    }
```

```
}  
return dp[n][k]
```

13.6

给定整数数组 $A[1 \dots n]$ ，要找到它的子数组 $A[i \dots j]$ ，使得 $A[i \dots j]$ 中各个元素的乘积最大

1. 假设 A 中的元素皆为正数，请给出相应的算法
如果皆为正数，正整数乘积是单调不减的，直接返回整个数组会得到正确答案。
2. 假设 A 中的元素有正有负，请给出相应的算法
 - 需要维护两个数组 `minF`, `maxF`，主要是考虑一个负的很大的数如果乘一个负数很可能变得很大，同理一个正的很大的数乘一个负数会变得很小，自然需要都考虑进转移方程。
 - 状态转移方程分别如下
 - `minF(i)=min{minF(i-1)*a_i,maxF(i-1)*a_i,a_i}`
 - `maxF(i)=max(minF(i-1))*a_i,minF(i-1)*a_i,a_i`
 - 单独乘一段不要忽略。
 - 最后返回 `max(maxF(i))` 即可

13.8

下面是与公共子序列相关的一系列问题：

1. 给定两个字符串 X 和 Y ， $X = \langle x_1, x_2, \dots, x_m \rangle$ $Y = \langle y_1, y_2, \dots, y_n \rangle$ 设计一个动态规划算法计算他们的最长公共子序列

用 $dp[i][j]$ 表达 X 的前 i 元素和 Y 的前 j 元素的最长公共子序列的长度。

- 如果 $X[i] == Y[j]$ ，则 $dp[i][j] = dp[i-1][j-1] + 1$ ，即这一位是公共的
- 否则，则可能需要考察 $i-1$, $j-1$ 和 i , j 不变的组合谁更长
- $dp[i][j] = \max\{dp[i-1][j], dp[i][j-1]\}$
初始化 $dp[0][0] = I_{x_1=y_1}$ ，计算依赖是依赖 i 更小， j 更小的，所以从上往下，从左往右计算。最后返回 $dp[m][n]$ 即可

2. 给定两个字符串 X 和 Y ，规定最长子序列中 X 的字符可以重复出现， Y 中的则不可以。

思考利用 1 中的算法：唯一的区别是，当 X 回退的时候，我们可以选择“不回退”，即重复利用上一个元素。这里会引入一个分歧判断，加一个语句即可。

即

- 如果 $X[i] == Y[j]$, 则 $dp[i][j] = \max\{dp[i-1][j-1] + 1, dp[i][j-1] + 1\}$

3. 除 X, Y 两个字符串外, 给定一个正整数 k , 字符串 X 的字符重复出现的次数不超过 k , Y 中的字符不可重复出现。

可以考虑一个简单策略: 我们可以把 X 字符串抄录成 X' , 对于 X 元素 x_i , 抄录 $\{x_i, x_i, x_i \dots\}$ (k 个), 然后直接应用算法 1.

也可以标记一个 `used[]` 记录使用次数, 在 2 基础上

13.9

请设计一个高效的算法, 找到字符串 $T[1 \dots n]$ 中前向和后向相同的最长连续子串的长度。前向和后向的子串不能重叠

规划子问题为 $dp[i][j]$, 其表示前向 i 位和后向从 n 到第 j 位的最长连续子串的长度

- 需要遵循 $i < j$, 保证不重叠
- 如果 $T[i] == T[j]$, 则说明 $dp[i][j] = dp[i+1][j-1] + 1$
- 否则, $dp[i][j] = \max\{dp[i+1][j], dp[i][j-1]\}$

计算顺序设计: i 依赖更大, j 依赖更小, 所以行从下向上, 行中从左到右

我们并不知道最后 i, j 停留在哪里, 但可以判断出答案一定满足 $i = j - 1$ (即不重叠的最好情况), 所以检查一下所有 $dp[i][i+1]$ 就可以得到答案

```
def maxSubStr(T):
    init dp[][]
    dp[i][j] that i>j =0
    for(int i=n-1;i>=1;i--){
        for(int j=i;j<=n;j++){
            if(T[i]==T[j]):
                dp[i][j]=dp[i+1][j-1]+1
            else:
                dp[i][j]=max(dp[i+1][j],dp[i][j-1])
        }
    }
    return max(dp[i][i+1] in all i);
```

13.10

令 $A[1 \dots m]$ 和 $B[1 \dots n]$ 是两个任意的序列。 A, B 的公共超序列 (supersequence) 是一个序列, 它包含 A 和 B 为其子序列。请设计一个高效算法找到 A, B 的最短公共超序列。

这个问题可以看作先找到最长公共子序列。然后将 AB 不在里面的部分按顺序加进去就好了。
具体来说怎么加？

- 找到 A 和 B 的最长公共子序列，将子序列元素在 A 和 B 中的下标记录 $T_1[] T_2[]$ ，这是两个长度和 C 相同的数组，其记录了 A 或 B 中某个元素在 $C[i-1]$ 之前的元素下标. 第一个记录为 0，
- 在找完公共子序列基础上，按下标将 A 和 B 的元素放进去，保持原有顺序
- SSS: shortest super sequence
- 找最长公共子序列的算法之前已经列过不再赘述，只需要维护 T_1 和 T_2 。

```
def SSS(A,B)
    find longest common sub sequence of A, B, that is C
    init D[]
    for i in len(C): # 从0开始，但是1才是C真正的元素，第一位是0
        for j in (T1[i],T1[i+1]):
            D.append(A[j])
        for k in (T2[i],T2[i+1]):
            D.append(B[k])
        D.append(C[i+1])
```

- 为什么这样就是最短超序列？
 - 可以证明：
 - 首先，最短公共超序列中一定包含最长公共子序列是一定的。这很显然。
 - 在建立完最长公共子序列 C 后，我们知道 $A-C$ 和 $B-C$ 一定没有重叠序列部分（否则最长公共子序列还可以延长，矛盾）
 - 所以只能将他们按顺序依次插入 C 中，这是最短的情况。

13.11

这个问题考虑最长公共子序列问题的两个变体。给定三个序列 X 、 Y 、 Z ，长度分别为 m, n, k ，现在的问题是判断序列 X 和 Y 能否合并成一个新的序列 Z ，不改变其中任何一个序列中元素的相对顺序。显然 $k = m + n$

假设有一人声称他找到了下面这个简单的算法。首先计算 X 和 Z 的 LCS，令 Z' 是从 Z 中将 LCS 中的元素删除得到的序列。如果 $Y = Z'$ ，则答案为是，否则为否。请分析上述算法是否正确，是则证明之，否则举反例。

不正确。

反例：

$$Z = ABACABC, X = ABC, Y = BACA$$

Z 可以构造出。如 $Z = \langle A_x, B_y, A_y, C_y, A_y, B_x, C_x \rangle$

$$LCS = A_x B_y C_y$$

$$\text{则 } Z' = Z - LCS = \langle A_y A_y B_x C_x \rangle$$

$Z' \neq Y$, 但是可以构造。所以矛盾

给出解决这个判定问题的 $O(mn)$ 的算法。（上一题正确与否都不能使用）

可以列一个表，大小为 mn ，在上面游走就是一个创造新串的过程。具体如下：

拿例题来说。

Y/X	A	B	C
B			
A			
C			
A			

建立一个数组 $dp[i][j]$ 记录表中 $[i, j]$ 位置块能构成的序列。

$$dp[i][j].append(dp[i-1][j] + Y[i]), dp[i][j].append(dp[i][j-1] + X[j])$$

特殊的： $dp[0][0]$ 存储 $X[0]Y[0]$ 和 $Y[0]X[0]$

规划计算顺序： i 从小到大， j 从小到大计算。即从原点开始游走，通过不同的路径到达终点（右下角）

计算：需要计算每一个格子的字符串，每一次计算格子读取当前 $Y[i], X[j]$ 和 $append$ 可以看作耗费 $O(1)$ ，总体计算即需要 $O(mn)$ 次

最后只需要 `return (Z in dp[m][n])? true : false` 即可

转化为优化问题。从 X, Y, Z 中删除最少的元素使得合并成立。时间复杂度为 $O(mnk)$

首先运行 2 中算法得到， X 和 Y 目前可以组合成的 Z ，增加一步标记 Z 中的元素来自 x 还是 y ，参照 1 中表示方式。这不会改变复杂度 $O(mn)$

然后，将 Z 和得到的 mn 个序列比对，具体来说和 Z' 比，得到 Z 和 Z' 的最长公共子序列长度 L 。用 Z 原长度减去 L 即所得，最后返回所有 Z' 能得到的最小值，总体是 $O(mnk)$

比对二者的最长公共子序列长度可以用

放弃了，不太懂，贴一个 AI 的回答，似懂非懂

我们采用三维动态规划解决此问题：

动态规划状态定义

定义一个三维数组 $dp[i][j][k]$ ，表示从 $X[0:i]$ 、 $Y[0:j]$ 和 $Z[0:k]$ 中删除的最少字符数，满足 $X[0:i]$ 和 $Y[0:j]$ 可以合并形成 $Z[0:k]$ 。

状态转移方程

1. 如果 $k = 0$ ，即目标序列为空，则删除的最少字符数是 $i + j$ ：

$$dp[i][j][0] = i + j$$

2. 如果 $i = 0$ 且 $j = 0$ ，即 X 和 Y 都为空，则

$$dp[0][0][k] = k \quad (\text{全部删除 } Z \text{ 的 } k \text{ 个字符})$$

3. 一般情况：

- 如果 $X[i-1] = Z[k-1]$ ，则考虑使用 $X[i-1]$ 匹配 $Z[k-1]$ ：

$$dp[i][j][k] = dp[i-1][j][k-1]$$

- 如果 $Y[j-1] = Z[k-1]$ ，则考虑使用 $Y[j-1]$ 匹配 $Z[k-1]$ ：

$$dp[i][j][k] = dp[i][j-1][k-1]$$

- 如果两者都不匹配，则 $Z[k-1]$ 无法由当前字符生成，需要删除 $Z[k-1]$ ：

13.12

给定包含 n 个字符的字符串 $s[1 \dots n]$ ，该字符串可能来自一本年代久远的书籍，只是由于纸张朽烂的缘故，文档中的所有标点符号都不见了 (例如 `itwasthebestoftimes...`)。现在你希望在字典的帮助下重建这个文档。在此，字典表示为一个布尔函数 $dict(\cdot)$ 。对于任意的字符串 w ,

$$dict(w) = \begin{cases} TRUE, & w \text{ is valid word} \\ FALSE, & \text{otherwise} \end{cases}$$

请给出一个动态规划算法，判断 $s[1 \dots n]$ 能否重建为合法单词组成的序列。假设调用 $dict$ 每次只需要一个单位的时间。时间复杂度要求不超过 $O(n^2)$

构建这样一个 $dp[i][k]$ 数组， i 表示从 0 到 i ， k 表示末尾一个单词的长度，总体 i 表示从 0 到 i 能否重建为由合法单词组成的序列

$$dp[i][0] = 0$$

递归方程为

- 如果 $dict(s[i-k+1:i])$ ，则 $dp[i][k] = dp[i-k][m]$ ，对某个 m 成立

- 用 `valid[i]` 存储前 i 位是否可行。
- $valid[i] = \bigwedge_k dp[i][k]$

```
def validWord(s):
    valid[0]=true;
    for(int i=1;i<=n;i++){
        valid[i]=false;
        for(k=1;k<=i;k++){
            if(dict(s[i-k+1,i]):
                #可以继续划分
                dp[i][k]==valid[i-k]
                valid[i]=true;
        }
    }
    return valid[n]
```

若是由合法单词序列构成的，请输出对应的单词序列

只需要在原算法中加入这样一个设计：记录每个 `valid[i]` 的合法情况。

```
def validWord2(s):
    valid[0]=true;
    words[][]
    words[0]=''
    for(int i=1;i<=n;i++){
        valid[i]=false;
        for(k=1;k<=i;k++){
            if(dict(s[i-k+1,i]):
                #可以继续划分
                dp[i][k]==valid[i-k]
                valid[i]=true;
                # 首先抄录依赖的words[i-k]
                words[i].extend(words[i-k])
                words[i].append(s[i-k+1])
        }
    }
    return valid[n],words[n]
```

13.13

当字符串颠倒和源字符串相同时，我们称为回文。

请给出一个找到给定字符串的满足回文条件的最长子序列的算法，算法最终只需要给出长度即可。

采取 dp 算法。

$dp[i][j]$ 表达从第 i 个字符到第 j 个字符的最长子序列回文

$dp[i][i] = 1$ 显然

$$dp[i][j] = \begin{cases} dp[i+1][j-1] + 2, & \text{if } A[i] == A[j] \\ \max\{dp[i+1][j], dp[i][j-1]\} & \\ 0, & i > j \end{cases}$$

```
def longest_palindromic_subsequence(S):
    n = len(S)
    dp = [[0] * n for _ in range(n)]

    # Base case: single character substrings
    for i in range(n):
        dp[i][i] = 1

    # Fill the dp table
    for length in range(2, n + 1): # Substring lengths from 2 to n
        for i in range(n - length + 1):
            j = i + length - 1
            if S[i] == S[j]:
                dp[i][j] = dp[i+1][j-1] + 2
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])

    return dp[0][n-1]
```

任何一个字符串都可以拆解成一组回文。请设计一个算法，计算对给定字符串可以拆分的最少回文数量，并分析算法的时间、空间复杂度

增加辅助函数，判断是否是回文

利用 $dp[i]$ ，其表示从第 i 位到末尾的最少回文数量

状态转移基于：

$dp[i] = \min(dp[i], 1 + dp[j + 1])$ ，如果 i 到 j 位是一个回文串

依赖关系是依赖更大的，所以 reverse 算

```
def min_palindrome_cuts(S):
    n = len(S)
```

```

# Step 1: Precompute isPalindrome table
isPalindrome = [[False] * n for _ in range(n)]
for i in range(n):
    isPalindrome[i][i] = True
for length in range(2, n + 1): # Length of the substring
    for i in range(n - length + 1):
        j = i + length - 1
        if S[i] == S[j]:
            if length == 2:
                isPalindrome[i][j] = True
            else:
                isPalindrome[i][j] = isPalindrome[i+1][j-1]

# Step 2: Compute dp array
dp = [float('inf')] * (n + 1)
dp[n] = 0 # Base case: empty substring requires no cuts
for i in range(n - 1, -1, -1): # Reverse order
    for j in range(i, n):
        if isPalindrome[i][j]: # i到j是回文, 则dp[i]是dp[j+1]情况再加一个串。
            dp[i] = min(dp[i], 1 + dp[j+1])

return dp[0]

```

计算 `isPalindrome` 的时间复杂度为 $O(n)$, 存储用了 $O(n^2)$ 计算 `dp[n]` 的过程耗费 $O(n^2)$ 时间,
 总体时间和空间复杂度都是 $O(n^2)$

13.15

考虑零钱兑换问题的各种变体

给定数量无限的面值分别为 $x_1, x_2, \dots, x_n$ 的硬币, 希望将金额 v 兑换成零钱, 但这有时候是不可能的。设计 $O(nv)$ 的动态规划算法, 判断能否兑换 v

根据提示, 可以通过金额和硬币面值的遍历来做。

用 $dp[i]$ 表示能否兑换面值 i

$dp[i] = \vee_j dp[i - x_j]$ 即遍历每一种面值, 来查 dp

初始化 $dp[0] = 1$, 即不用钱就可以兑换

最后返回 $dp[v]$ 即可

每一次 $dp[i]$ 的计算要遍历一次硬币 n 的数组 `dp[i]` 要计算 v 次, 所以是 $O(nv)$

考虑上述的变体: 每种面值的硬币最多只能用一次。设计 $O(nv)$ 算法, 判断能否兑换

只需要更改一下。

首先在多存一个量 `coins[]`，代表零钱使用

```
def exchangePro(X,v):
    dp[0]=1;
    coins=[0]*n
    for(int i=1;i<=v;i++){
        for j in range (1,n):
            if(dp[i-x_j]==1):
                dp[i]=dp[i-x_j]
                # 记录零钱使用情况,即在上一个可行解的基础上,扩充当前金额
                coins[i].append(coins[i-x_j])
                coins[i].append(x_j)
    }
    return coins[v],dp[v]
```

13.16

图 $G = (V, E)$ 的一个顶点覆盖 S 是 V 的子集，满足： E 的每一条边都至少有一个端点属于 S 。请给出如下问题的一个线性时间的算法：- 输入：无向树 $T = (V, E)$ ，输出： T 的最小顶点覆盖的大小

我们用 DFS 从根出发遍历树，对于每一个遍历到的点 v ，有两种选择：

- 将其加入最小顶点覆盖
- 不加入

我们用两个数组，`select[]` 和 `ignore[]` 记录一个点选与不选，其在遍历中的最优答案。由于独特的树结构，所以如果 $ignore[u]$ ，则所有 uv 边只能靠 v 选中。

$$select[u] = \sum_v \min\{select[v], ignore[v]\} + 1$$
$$ignore[u] = \sum_v select[v]$$

where uv 是一条边，即 v 是 u 的孩子

初始化所有叶子节点 `select[leaf]=1, ignore[leaf]=0`

最终返回答案 $\min\{select[root], ignore[root]\}$ 即可。

具体而言，用 DFS 做，后序遍历（保证操作时，孩子们已经被赋值）

```
initialize all color to white
def minVCoverDFS(V,E,v):

    select[v]=0; ignore[v]=0
```

```

for all w relate to v:
    if w is white:
        w.color=gray
        minVCoverDFS(V,E,w)

select[v]+=1
for all w relate to v:
    select[v]+= min(select[w],ignore[v])
    ignore[v]+=select[w]
v.color = black

```

13.18

假设你准备开始一次长途旅行。以 0 公里为起点，一路上一共有 n 座旅店，距离起点的公里数分别为 $a_1 < a_2 < \dots < a_n$ 。旅途中，你只能在这些旅店停留。最后一座旅店 a_n 为你的终点。理想情况下，你每天可以行进 200 公里，不过考虑到旅店间的实际距离，有时候可能还达不到这么远。如果你某天走了 x 公里，你将受到 $(200 - x)^2$ 的惩罚。你需要计划好行程，使得总惩罚最小。设计一个高效的算法，计算一路上的最优停留位置序列

- 每个旅店都可以选择停留，或者不停留
- 每个旅店选择停留或不停留会影响我的行进距离计算（跳过某旅店，则需要继续走，不能超过 200km，停留则重新计算 200km）
- 引入 $dp[i]$ 表达到达第 i 个旅店时候的最优惩罚，则 $dp[i]$ 依赖于是从哪个旅店到达
- 即

$$dp[i] = \min\{dp[j] + (200 - (a_i - a_j))^2 \mid \text{if } a_i - a_j \leq 200\}$$

```

def min_penalty(a):
    n = len(a) # 旅店数量
    dp = [float('inf')] * n # dp数组，存储最小惩罚
    path = [-1] * n # 路径数组，存储最优停留序列
    dp[0] = 0 # 起点惩罚为0

    for i in range(1, n):
        for j in range(i):
            cost = (200 - (a[i] - a[j])) ** 2
            if dp[j] + cost < dp[i]:
                dp[i] = dp[j] + cost

```

```

        path[i] = j

# 通过路径数组恢复最优停留位置序列
stops = []
current = n - 1
while current != -1:
    stops.append(current)
    current = path[current]
stops.reverse()

return dp[-1], stops

# 示例
a = [0, 100, 250, 370, 480] # 旅店位置
min_penalty_cost, stops = min_penalty(a)
print("最小惩罚:", min_penalty_cost)
print("最优停留序列:", stops)

```

13.20

现有一个加油站，它的地下油库能存储 L 升的油。每次进油有固定的代价 P 。当第一天的油未用完时，每升油存储 1 天的代价为 c 。假设在冬天歇业之前，加油站还要运营 n 天。第 n 天结束时，油库里的油必须全部卖完。假设根据历史数据，加油站可以精准地预知未来 n 天每天的油的销售量 $g_i (1 \leq i \leq n)$ 。第 0 天结束时，油库是空的。请设计一个算法，决定未来 n 天的进油计划，使得总代价最小。

由于最终来看，油能完整满足某一天的需求是更优的选择，在这基础上，我们可以决定在过去的某一天 j 进油，满足一直到 i 天

设计一个 $dp[i]$ 表示第 i 天的最低总代价。

$dp[i]$ 依赖于 $dp[j], j < i$ ，即 j 天选择进油的话，则需要存储 $\sum_{j \leq i} g_j$

转移方程

$$dp[i] = \min_j \left\{ dp[j] + P + c \sum_{j \leq i} g_j \times (\text{天数}) \right\}$$

```

def min_cost(n, L, P, c, g):
    # dp 数组，存储最小总代价
    dp = [float('inf')] * (n + 1)
    dp[0] = 0 # 初始条件

```

```

# 前缀和数组，用于快速计算需求
prefix_sum = [0] * (n + 1)
for i in range(1, n + 1):
    prefix_sum[i] = prefix_sum[i - 1] + g[i - 1]

for i in range(1, n + 1):
    for j in range(1, i + 1):
        # 第 j 天进油，满足从第 j 天到第 i 天的需求
        needed_oil = prefix_sum[i] - prefix_sum[j - 1]
        if needed_oil <= L: # 进油量不能超过油库容量
            # 计算存储成本
            storage_cost = 0
            for k in range(j, i):
                storage_cost += (prefix_sum[k] - prefix_sum[j - 1]) * c
            dp[i] = min(dp[i], dp[j - 1] + P + storage_cost)

return dp[n]

```

13.23

假设你需要为一家公司策划一次宴会。公司所有人员组成一个层级关系，即所有人按照上下级关系组成一棵树，树的根节点为公司董事长。公司的人力资源部门为每个员工评估了一个实数值的友好度评分。为了聚会能够更轻松的进行，公司不希望一名员工和他的直接领导共同出现在宴会上。请设计一个算法决定宴会邀请人员名单，使得所有参加人员的友好度评分总和最大。

- 仍然引入 `ignore[]` 和 `select[]` 两个数组，作为选择/忽略某个节点的最优友好度评分总和
- for a tree edge uv , when u is selected, then v can't be selected. if u is ignored, then v can be selected or not.
- We want `root` as the return of answer. So it has to know all circumstance. One parent node's `select` and `ignore` rely on its children
- that is

$$\begin{aligned}
 ignore[u] &= \sum_v \max\{ignore[v], select[v]\} \\
 select[u] &= \sum_v ignore[v] + friend[u]
 \end{aligned}$$

可以看到我们的算法本质是依赖父亲节点的选择的。

```

init all color to white
def friendDP(friendly[], root):
    for all leaf node do:
        select[leaf]=friendly[leaf]
        ignore[leaf]=0
        all other select and ignore initilize to 0
    friendDFS(root)
    return max(select[root], ignore[root])

def friendDFS(G, v, friendly):
    for all neighbor w of v:
        w.color = gray
        friendDFS(w)

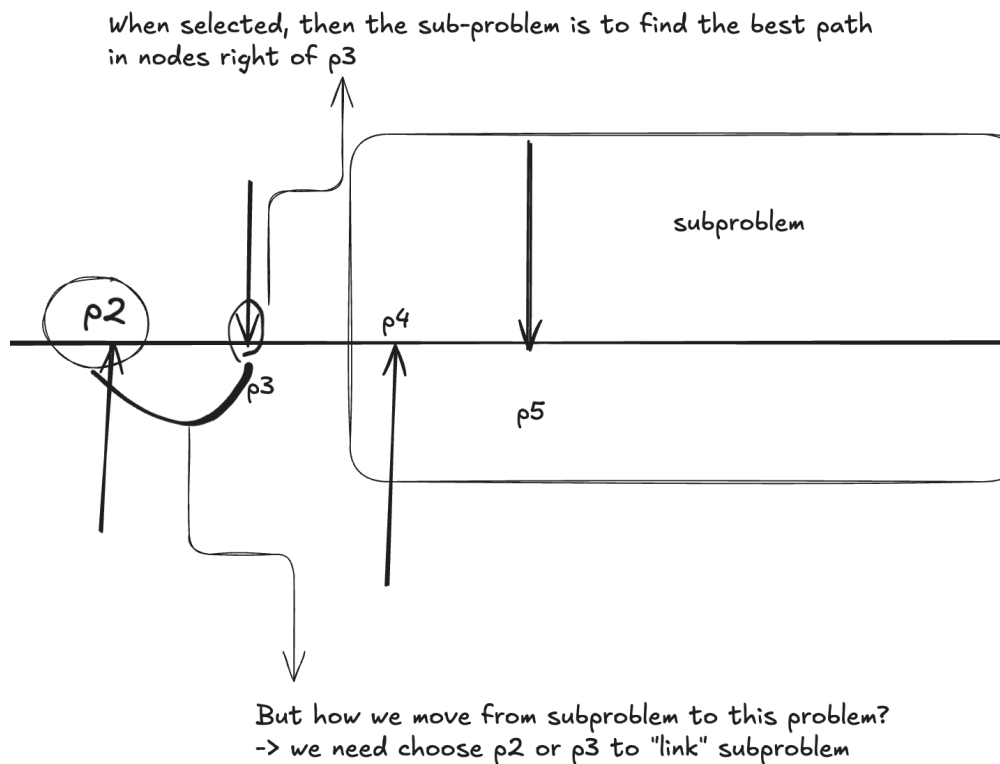
    ignore[v]=SUM max(ignore[w], select[w]) for all w
    select[v]=SUM ignore[w]+friendly[v]
    v.color = black
    return

```

13.24

一个送披萨的男孩有一系列的订单要送。这些订单的目的地用坐标 $\{p_1, p_2, \dots, p_n\}$ 表示。 $p_i = (x_i, y_i)$ 。假设 x 坐标是严格递增的，即 $x_1 < x_2 < \dots < x_n$ 。比萨店的位置在 p_1 ，并且送外卖的路线满足两个约束条件：首先按照 x 坐标递增的顺序送外卖，然后按照 x 递减的顺序送外卖并且最终回到比萨店。假设已有函数 $dist(i, j)$ 用来计算 p_i 和 p_j 之间的距离。请给出计算最短外卖路线的算法。

- 考虑子问题划分：



每个点可以选择：

- 作为去方向
- 作为回方向

使用 $Go[]$ 作为去方向时，该点到终点的最优去代价总和。

使用 $Back[]$ 作为回方向时，从该点到终点的最优回代价总和

所以我们的最终最优路线长度，就等于 $Go[root] + Back[end]$

- 且 $root$ 和 end 都是唯一确定的

子问题递归：

- 同时，由于去向点和回向点可能冲突，也要考虑进我们的遍历。
- 如果一个点选择作为去方向，则其依赖右侧的点，

$$Go[i] = \min_{j>i} \{Go[j] + dist[i, j]\}$$

- 如果一个点选择作为回方向，

$$Back[i] = \min_{j>i} \{Back[j] + dist[i, j]\}$$

在局部选择某个点作为去向还是回向的最优问题。

- 如果 $Back[i] < Go[i]$ 选择将 i 的方向记录为 $Back$ ，同时将 $Go[i]$ 改为 inf （即不让他再作为 Go 边）
- 反之亦然。

最后返回 $Go[root] + Back[root]$

计算顺序：从 i 较大的开始做。

#TODO 有点难。不太知道如何划分子问题了.